

Assessment Tool 3 — Internet Programming (CSA43)

CO-2: Short Answer Test — Full Solutions

Question 1 (5 Marks): Responsive Web Design

Scenario: A company wants its website to work seamlessly on mobile, tablet, and desktop devices.

Concept and importance of responsive web design

Responsive web design (RWD) is an approach to building webpages so that a **single codebase automatically adapts** its layout, images, and typography to the size and orientation of the device viewing it — instead of maintaining separate sites for mobile and desktop. It is important because:

- The majority of web traffic today comes from a mix of phones, tablets, and desktops with widely varying screen widths.
- It improves **usability** (no pinch-zooming, no broken layouts) and **accessibility**.
- Search engines like Google use **mobile-friendliness** as a ranking factor, so RWD directly affects SEO.
- It reduces development and maintenance cost compared to running separate “m.site.com” mobile versions.

Key CSS techniques used to achieve responsiveness

1. **Viewport meta tag** — `<meta name="viewport" content="width=device-width, initial-scale=1.0">` tells mobile browsers to render at the device's actual width.
2. **Fluid/relative units** — %, em, rem, vw/vh, fr instead of fixed px, so elements scale with the container/viewport.
3. **Media queries** — `@media (max-width: ...) / (min-width: ...)` apply different CSS rules at different breakpoints.
4. **Flexible images** — `img { max-width: 100%; height: auto; }` so images shrink with their container.
5. **CSS Flexbox and Grid** — layout systems that redistribute space and reflow items automatically.
6. **box-sizing: border-box** — keeps padding/border from breaking declared widths across screen sizes.

Using Flexbox or Grid for adaptive layouts

Flexbox arranges items along a single axis (row or column) and is ideal for components like navigation bars, where items should distribute or wrap along one direction:

```
.navbar { display: flex; justify-content: space-between; flex-wrap: wrap; }
```

Grid arranges items in two dimensions (rows *and* columns simultaneously) and is ideal for page-level skeletons and card layouts:

```
.page {  
  display: grid;  
  grid-template-columns: 220px 1fr;  
  gap: 1rem;  
}
```

Example layout

```
<div class="container">  
  <header class="header">Site Header</header>  
  <main class="content">Main Content</main>  
  <aside class="sidebar">Sidebar</aside>  
  <footer class="footer">Footer</footer>  
</div>  
  
.container {  
  display: grid;  
  grid-template-columns: 3fr 1fr;  
  grid-template-areas:  
    "header header"  
    "content sidebar"  
    "footer footer";  
  gap: 1rem;  
}  
.header { grid-area: header; } .content { grid-area: content; }  
.sidebar { grid-area: sidebar; } .footer { grid-area: footer; }  
  
@media (max-width: 700px) {  
  .container {  
    grid-template-columns: 1fr;  
    grid-template-areas: "header" "content" "sidebar" "footer";  
  }  
}
```

On desktop this shows a 2-column layout (main content + sidebar); below 700px, grid-template-areas collapses everything into a single stacked column.

Suggested improvements

1. Use **clamp()** for fluid typography so text scales continuously instead of jumping at breakpoints.
2. Adopt **CSS Container Queries** so components adapt to their container's size, not just the viewport.
3. Test with real devices/emulators (Chrome DevTools, BrowserStack) rather than only resizing the desktop browser window.

4. Use `srcset/sizes` on images to serve device-appropriate resolutions, improving both layout and load performance.
-

Question 2 (5 Marks): Client-Side Form Validation

Scenario: A webpage contains a form where user input must be validated before submission.

Concept and purpose of client-side validation

Client-side validation checks user input **in the browser, before the form is submitted** to the server. Its purposes are:

- Giving the user **immediate feedback** on mistakes (missing fields, wrong format) without a round-trip to the server.
- Reducing unnecessary **server load** by filtering out obviously invalid submissions early.
- Improving **user experience**, since errors are shown right where and when they occur.
- Note: it is **not a security measure** by itself — it must always be paired with server-side validation, since client-side JavaScript can be bypassed or disabled.

JavaScript event handling in validation

Validation logic is attached to DOM events so it runs at the right moment:

- **input** — fires on every keystroke; used for real-time, as-you-type feedback.
- **blur** (or **focusout**) — fires when a field loses focus; commonly used to validate a field once the user has finished with it.
- **submit** — fires when the form is submitted; used for a final, complete validation pass and to block submission with `event.preventDefault()` if invalid.

```
form.addEventListener('submit', (e) => {  
  e.preventDefault();  
  if (!isValid) return;  
  form.submit();  
});
```

Role of regular expressions

Regular expressions (regex) describe a **pattern** that valid input must match, letting JavaScript test structured formats like email addresses or phone numbers in one line:

```
const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;  
const phoneRegex = /^[6-9]\d{9}$/;  
  
emailRegex.test("user@example.com"); // true  
phoneRegex.test("9876543210"); // true
```

Regex catches structural errors (missing @, wrong digit count) that a simple “not empty” check would miss.

Example: real-time email validation

```
<input type="email" id="email" required />
<span id="emailError"></span>

const email = document.getElementById('email');
const error = document.getElementById('emailError');

email.addEventListener('input', () => {
  error.textContent = emailRegex.test(email.value)
    ? ''
    : 'Enter a valid email address';
});
```

Suggested improvements

1. Always **re-validate on the server**, since client-side checks can be bypassed.
 2. **Debounce** the input listener so error messages don't flash while the user is still typing.
 3. Add **ARIA attributes** (aria-invalid, aria-describedby) for screen-reader accessibility.
 4. Use libraries like libphonenumber-js for robust international phone validation instead of a single fixed regex.
 5. Rate-limit/CAPTCHA the backend endpoint to guard against automated/bot submissions.
-

Question 3 (5 Marks): CSS Box Model, Positioning, and Layout Stability

Scenario: A webpage layout breaks when the browser window is resized.

The CSS box model and its components

Every HTML element is rendered as a rectangular box made of four layers, from the inside out:

1. **Content** — the actual text/image/element.
2. **Padding** — space between the content and the border (inside the element's background).
3. **Border** — a visible (or invisible) line around the padding.
4. **Margin** — space *outside* the border, separating the element from its neighbors.

By default (box-sizing: content-box), width/height apply **only to the content**, so padding and border are added on top, making the rendered box larger than the declared size. Setting box-sizing: border-box makes width/height include padding and border, giving predictable sizing — this is a common fix for layouts that “break” at certain widths.

How CSS positioning affects layout structure

- **static** (default) — elements flow normally in document order.
- **relative** — element stays in normal flow but can be visually offset (top/left) without affecting siblings.
- **absolute** — removed from normal flow entirely; positioned relative to the nearest **positioned** ancestor (or the viewport if none exists) — a frequent cause of unexpected overlap if that ancestor isn't set correctly.
- **fixed** — positioned relative to the viewport and stays in place during scrolling.
- **sticky** — behaves as relative until a scroll threshold, then “sticks” like fixed.

Misusing absolute/fixed positioning without properly establishing the containing block is one of the most common reasons layouts break when the window resizes.

Possible causes of layout breaking on resize

- Fixed px widths instead of fluid units (% , fr , minmax()).
- box-sizing: content-box combined with padding/border, making true widths exceed declared widths.
- Floated elements without clear, or without flex-wrap/wrapping equivalents.
- position: absolute elements anchored to the wrong ancestor.
- No media queries to restructure the layout at smaller widths.
- Fixed-height containers that clip content once text reflows onto more lines.

How media queries resolve these issues

Media queries apply different CSS rules based on the viewport's width (or other features), letting the layout **restructure at defined breakpoints** rather than trying to force one fixed layout onto every screen size:

```
.layout { display: flex; flex-wrap: wrap; gap: 1rem; }
.sidebar { flex: 1 1 250px; }
.main    { flex: 3 1 600px; }

@media (max-width: 700px) {
  .layout { flex-direction: column; }
}
```

Here, flex-wrap: wrap already lets items reflow naturally, and the media query goes further by explicitly stacking the layout vertically below 700px for a cleaner mobile reading order.

Best practices for a stable, responsive layout

1. Apply `* { box-sizing: border-box; }` globally as a baseline reset.
2. Prefer **Flexbox/Grid** over floats and manual absolute positioning for structural layout.
3. Use **relative units** and minmax()/clamp() instead of fixed pixel widths.

4. Add flex-wrap: wrap or responsive grid-template-columns so content reflows instead of overlapping.
 5. Test resizing continuously in DevTools' responsive mode across common breakpoints during development, not only at the end.
-

Question 4 (5 Marks): DOM Manipulation for Dynamic Content

Scenario: A website requires dynamic content updates without reloading the entire page.

Concept of the Document Object Model (DOM) and its role

The DOM is a **tree-structured, in-memory representation** of an HTML document that the browser builds when it parses a page. Every HTML tag becomes a **node** in this tree (elements, attributes, text), and JavaScript can read or modify these nodes directly. The DOM's role in web development is to act as the **programming interface** between the static HTML markup and dynamic, interactive behavior — it is what lets JavaScript “see” and change what's on the page after the initial load, enabling features like live search results, form feedback, and single-page-application-style updates without a full page reload.

How JavaScript manipulates the DOM

Common DOM manipulation operations:

- **Selecting elements:** `document.querySelector()`, `document.getElementById()`.
- **Changing content:** `element.textContent`, `element.innerHTML`.
- **Changing attributes/styles:** `element.setAttribute()`, `element.classList.add()`, `element.style.property`.
- **Creating/removing nodes:** `document.createElement()`, `element.appendChild()`, `element.remove()`.
- **Responding to interaction:** `element.addEventListener('click', handler)`.

Example: updating content dynamically

```
<button id="loadBtn">Load More</button>
<ul id="itemList"></ul>

const btn = document.getElementById('loadBtn');
const list = document.getElementById('itemList');
let count = 0;

btn.addEventListener('click', () => {
  count++;
  const li = document.createElement('li');
  li.textContent = `Item ${count}`;
  list.appendChild(li);
});
```

Each click **creates a new node and inserts it into the existing **, updating the visible page instantly — the browser does not reload or re-fetch the whole HTML document.

Suggested improvements for performance and UX

1. **Batch DOM updates** — build content in a DocumentFragment and append it once, instead of calling `appendChild` repeatedly, to minimize reflows/repaints.
2. Use **event delegation** (one listener on a parent, checking `event.target`) instead of attaching a listener to every dynamically created child element.
3. Prefer **textContent** over `innerHTML` when inserting plain text, to avoid unnecessary HTML parsing and XSS risk from unsanitized input.
4. For frequent/large updates, consider a **virtual-DOM library** (React, Vue) which diffs changes and updates only what's necessary, rather than manual DOM manipulation at scale.
5. Add **loading indicators** and **debounce** rapid-fire updates (e.g., live search) so the UI stays responsive and doesn't flood the DOM with updates.

Code of Conduct

I **MANDLA VISWATEJA** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____